VNUHCM –UNIVERSITY OF SCIENCE

ADVANCED PROGRAM IN COMPUTER SCIENCE

Project Report

CS418 - Introduction to Natural Language Processing

22TT2 –Group 30

22125010 –Phan Phúc Bảo
22125095 - Đào Xuân Thành
22125096 –Đoàn Công Thành
22125103 –Nguyễn Võ Hoàng Thông

Contents

1 Introduction

Natural language processing (NLP) tasks related to SinoNom (the Vietnamese old script based on the Chinese script system) and Chinese text present a rich area of study due to the intricate linguistic, cultural, and historical nuances embedded in these writing systems. These challenges make such tasks both intellectually stimulating and technically demanding. One particularly compelling task is prose-verse classification, which involves distinguishing between prose and verse in textual content. This task holds practical value as it can guide readers, especially those less proficient in Chinese or SinoNom, by improving the clarity and accessibility of the texts.

In this project, we explore the problem of prose-verse classification through a multi-faceted approach. Our primary focus is on leveraging machine learning techniques, employing two different pre-trained models to analyze and classify textual data. Additionally, we incorporate a rule-based method as a complementary reference, grounded in the structural characteristics of widely recognized ancient and modern Vietnamese verse genres and prose. By integrating these approaches, we aim to provide a comprehensive exploration of the problem domain.

To rigorously assess the effectiveness of our methods, we conduct a series of experiments to empirically evaluate the performance of each approach. These experiments allow us to compare the strengths and limitations of machine learning-driven models against the manually crafted rule-based algorithm. Furthermore, this project serves as a learning opportunity for our team, enabling us to deepen our understanding of machine learning methodologies and refine our ability to apply these techniques to a practical, linguistically significant problem.

2 Dataset

2.1 Data Collection

The dataset includes a variety of Chinese and SinoNom texts, sourced from classical and modern literature, ensuring a balanced representation of prose and verse. Prose samples typically include narratives, essays, and historical texts, while verse samples consist of poetry and other rhythmic or metrical compositions. Each sample is labeled as either prose or verse to facilitate supervised learning and evaluation. Table 1 shows different sources in the dataset.

No	Name	Type	Number of data items
1	Thuy Hu	Prose	7508
2	Dai Viet Su Ki Toan Thu	Prose	5500
3	Tam Quoc Dien Nghia	Prose	5401
4	Kieu	Verse	298
5	Mixture of verse from professor's data	Verse	16620
Total			35327

Table 1: Different sources for training and testing dataset with number of data items after preprocessing

2.2 Dataset Summary

Three prose sources including Thuy Hu, Dai Viet Su Ki Toan Thu, and Tam Quoc Dien Nghia provide a combined total of 18,409 data items. These are significant historical and literary works, ensuring comprehensive coverage of prose styles.

Verse is represented by two sources: Kieu and Mixture of verse from professor's data, totaling 16,918 items. The variety in verse sources captures different poetic forms and styles, which will help the model generalize better.

The dataset contains 35,327 items, with contributions from both prose and verse categories. This substantial volume of data ensures a robust foundation for training and testing the recognition model.

3 Methodology

3.1 Rule-Based Approach

3.1.1 Specification

This section outlines a rule-based methodology for classifying texts as either prose or verse, with a secondary focus on identifying specific genres of verse. The primary objective is binary classification—distinguishing prose from verse—while the secondary objective leverages structural patterns to categorize verse into traditional Vietnamese poetic forms. By systematically applying predefined rules, this approach combines interpretability and domain expertise to address the classification task.

The process begins with text preprocessing, where the input is segmented into lines, and extraneous whitespace is removed to ensure consistency. Line lengths are calculated to create a standardized dataset for analysis. These lengths form the foundation for applying rules that detect structural patterns indicative of specific verse genres.

The classification relies on detecting line-based patterns unique to Vietnamese verse forms. For example, texts with exactly four lines of seven characters each are classified as "Thất Ngôn Tứ Tuyệt," while texts with eight lines of the same length are identified as "Thất Ngôn Bát Cú." Another common form, "Lục Bát," is detected using a sliding window technique that examines groups of four consecutive lines. This technique checks for the alternating 6-8-6-8 pattern characteristic of the genre. Similarly, the "Song Thất Lục Bát" form, consisting of two 7-character lines followed by one 6-character line and one 8-character line, is identified using the same sliding window approach to locate the 7-7-6-8 sequence.

In addition to these specific genres, the methodology incorporates generalized classifications to handle variability in poetic forms. Texts with line lengths consistently falling within a defined range, such as 4 to 8 characters, are classified based on their minimum and maximum lengths, yielding descriptions like "4-8-character verse." For texts that deviate from traditional structures but still exhibit some regularity, the approach includes a semi-free verse category. This classification identifies sequences of lines with consistent lengths and calculates the proportion of the most frequently occurring line length. If this proportion exceeds a defined threshold (defaulting to 0), the text is labeled as semi-free verse. Texts that fail to conform to any recognized patterns are categorized as free verse, ensuring every input receives a classification.

The binary classification of prose versus verse builds on this genre-level analysis. Texts failing to match any predefined verse structure are presumed to be prose, based on the assumption that prose lacks the structural regularities inherent in verse. Preprocessing further aids this analysis by replacing Chinese punctuation with line breaks, enabling a line-by-line evaluation of structure. This approach ensures that even texts with ambiguous characteristics can be systematically assessed.

The methodology offers several technical advantages. The use of explicit rules and structural patterns ensures transparency and interpretability, making it easier to understand and refine. By integrating domain knowledge of Vietnamese verse forms, the approach effectively leverages linguistic expertise. The inclusion of generalized and semi-free verse classifications enhances flexibility, accommodating texts that do not strictly adhere to traditional genres.

However, the method is not without limitations. Its reliance on predefined patterns restricts its ability to handle unconventional or hybrid text forms. Additionally, the classification of prose as a default for texts failing to match verse structures may overlook subtle distinctions between loosely structured verse and prose.

Overall, this rule-based methodology provides a robust and interpretable framework

for distinguishing prose from verse while offering detailed insights into verse genres. It serves as both a standalone tool for text classification and a baseline for evaluating more advanced machine learning models, contributing to a deeper understanding of the structural features in SinoNom and Chinese texts.

3.1.2 Code Interpretation

The algorithm's functionality centers on two main tasks: preprocessing input text to preserve structural integrity and systematically classifying it as prose or verse using predefined rules.

```python
def clean_text(text):
    """
    Better preprocessing for text that preserves verse structure.
    """
    # Remove spaces while preserving original line breaks
    text = ''.join(text.split())

    # Standardize line breaks
    text = text.replace('。。。', '…').replace('...', '…')  # Handle ellipsis

    # Only split on actual line breaks and specific end-of-verse punctuation
    text = text.replace('。', '\n').replace('！', '\n').replace('？', '\n')

    # Remove other punctuation that shouldn't affect verse structure
    text = text.replace('，', '').replace('、', '').replace('；', '')
    text = text.replace('：', '').replace('「', '').replace('」', '')
    text = text.replace('"', '').replace('"', '').replace('…', '')

    # Clean up multiple newlines and empty lines
    lines = [line.strip() for line in text.split('\n')]
    lines = [line for line in lines if line]

    return '\n'.join(lines)
```

Listing 1: Function to clean text

```python
def classify_verse(text):
    """
    Improved verse classification with better structure detection.
    """
    # Clean and split text
    lines = [line.strip() for line in text.split("\n") if line.strip()]
    line_lengths = [len(line) for line in lines]

    if not lines:
        return "prose"  # Empty text is considered prose

    # Minimum length requirement for verse classification
    if len(lines) < 2:
        return "prose"

    # Check for Thất ngôn tứ tuyệt (7-character, 4 lines)
    if len(lines) == 4 and all(length == 7 for length in line_lengths):
        return "Thất ngôn tứ tuyệt"
```

```
    # Check for Thất ngôn bát cú (7-character, 8 lines)
    if len(lines) == 8 and all(length == 7 for length in line_lengths):
        return "Thất ngôn bát cú"

    # Check for Lục Bát pattern
    luc_bat_pattern = True
    if len(lines) >= 4 and len(lines) % 2 == 0:
        for i in range(0, len(lines), 2):
            if i + 1 < len(lines):
                if not (line_lengths[i] == 6 and line_lengths[i + 1] == 8):
                    luc_bat_pattern = False
                    break
        if luc_bat_pattern:
            return "Lục Bát"

    # Check for Song Thất Lục Bát
    if len(lines) >= 4 and len(lines) % 4 == 0:
        for i in range(0, len(lines), 4):
            if i + 3 < len(lines):
                segment = line_lengths[i:i + 4]
                if segment == [7, 7, 6, 8]:
                    return "Song Thất Lục Bát"

    # Check for regular character-count verses
    char_counts = set(line_lengths)
    if char_counts.issubset({4, 5, 6, 7, 8}):
        # If more than 80% of lines have the same length, it's a regular verse
        most_common_length = max(set(line_lengths), key=line_lengths.count)
        if line_lengths.count(most_common_length) / len(line_lengths) >= 0.8:
            return f"{most_common_length}-character verse"
        return f"{min(line_lengths)}-{max(line_lengths)}-character verse"

    # Semi-Free Verse detection
    sequence_counts = defaultdict(int)
    current_length = line_lengths[0]
    count = 1

    for length in line_lengths[1:]:
        if length == current_length:
            count += 1
        else:
            sequence_counts[current_length] += count
            current_length = length
            count = 1
    sequence_counts[current_length] += count

    total_sequences = sum(sequence_counts.values())
    most_common = max(sequence_counts.values())

    # More stringent criteria for Semi-Free Verse
    if most_common / total_sequences >= 0.4:  # At least 40% consistent pattern
        return "Semi-Free Verse"

    return "Free verse"
```

Listing 2: Function to preprocess text

Text Preprocessing with `clean_text`

The `clean_text` function enhances the preprocessing pipeline by addressing specific characteristics of Chinese and Vietnamese text:

```python
def detect_prose_verse(text):
    cleaned_text = clean_chinese_text(text)
    verse_type = classify_verse(cleaned_text)

    # Consider as prose if:
    # 1. It's free verse with very long lines
    # 2. It has very few lines
    # 3. Lines are too variable in length
    lines = cleaned_text.split('\n')
    if verse_type == "Free verse":
        if any(len(line) > 20 for line in lines):  # Very long lines suggest prose
            return "prose"
        if len(lines) < 3:  # Too few lines suggest prose
            return "prose"

    return "verse" if verse_type != "Free verse" else "prose"
```

Listing 3: Function to detect prose or verse

- **Line Breaks and Punctuation:** It replaces punctuation marks (⌈。⌋ , ⌈!⌋ , ⌈?⌋) with line breaks, aligning with the structural delineations of traditional verse. This step ensures that line-based analysis reflects the original segmentation intended by the text's composition.
- **Ellipsis Normalization:** Variations of ellipses (e.g., ⌈。。。⌋ , . . .) are standardized to a single form (⋯), minimizing inconsistencies in line length calculation.
- **Whitespace and Empty Lines Removal:** Spaces and unnecessary blank lines are stripped to maintain structural purity while preserving meaningful content.

This preprocessing step not only standardizes input but also prepares the data for reliable rule application in subsequent steps.

Verse Classification with `classify_verse`

The `classify_verse` function encapsulates the core logic for identifying specific verse genres and broader categories:

1. **Genre-Specific Patterns:** The function applies direct checks for traditional Vietnamese poetic forms, such as *Thất Ngôn Tứ Tuyệt*, *Thất Ngôn Bát Cú*, *Lục Bát*, and *Song Thất Lục Bát*. These checks rely on fixed rules for line counts and lengths, leveraging domain-specific knowledge to ensure precision.
2. **Regular and Semi-Free Verse Detection:**
   - **Regular Verse:** Line lengths falling within a limited set ({4, 5, 6, 7, 8}) are analyzed for consistency. A threshold of 80% consistency is applied to classify texts with predominantly uniform lengths as regular verse.
   - **Semi-Free Verse:** The detection algorithm uses a frequency analysis of line lengths, comparing the proportion of the most common length to the total

sequence. A threshold of 40% ensures that semi-free verse classifications reflect a balance between structural regularity and flexibility.

3. **Fallback Classification:** Texts that fail to meet any defined criteria default to *Free verse.* This inclusive category ensures comprehensive classification coverage.

Prose Detection with `detect_prose_verse`

The `detect_prose_verse` function extends the classification logic to distinguish prose:

- **Enhanced Line-Based Analysis:** The function considers factors like line count, variability, and maximum length to refine prose detection. For instance, texts with excessively long lines or too few lines are likely to be prose, reflecting the structural tendencies of such texts.
- **Iterative Evaluation:** By combining the output of `classify_verse` with additional prose-specific heuristics, this function offers a balanced assessment that accommodates ambiguous cases.

3.1.3 Design and Implementation Considerations

- **Interpretability and Modular Design:** The modular structure of the code, with distinct functions for preprocessing, classification, and detection, enhances readability and ease of maintenance.
- **Domain-Specific Adjustments:** The rules and thresholds are tailored to Vietnamese verse traditions, reflecting careful integration of cultural and linguistic expertise.
- **Flexibility for Refinement:** The use of adjustable thresholds (e.g., 40% for semi-free verse) ensures adaptability for future improvements.

3.2 Model-Based Approach

This section presents a model-based method to classify SinoNom/Chinese texts as prose or verse. We used pre-trained language models and fine-tune them to work with our data. Since the rule-based method is limited by requiring punctuation and spaces, the model-based method tried to uncover patterns and meanings from the data. Clearly, this approach is promising than rule-based approach for handling a wide range of text types and unclear cases, making it a better solution for the classification task.

3.2.1 Dataset Preprocessing for Fine-Tuning

Our data preprocessing task focused on optimizing the input format for fine-tuning models. We tried to preserve the semantic integrity of the text. For prose and verse, we implemented different preprocessing strategies due to their distinct structural characteristics:

- **Prose processing**: We split the data into items with smaller parts, with the requirement that the minimum of each data item is 50 characters, and an data item must contain complete sentences. This ensures that the model can process relative semantic units while maintaining computational efficiency.
- **Verse processing**: Verse, however, cannot be processed in the same way as prose, because doing so would fail to preserve the meaning of the verse. A verse must be treated as a complete unit to retain its full meaning. Therefore, we decided to treat each verse as a single data item. Since most verses in our dataset contain a small number of characters, we can minimize the truncation problem (i.e., the issue where the model cannot process the entire input of a data item).

For **general processing**, we removed all punctuation and spaces from each data item to allow the model to focus solely on the characters. This approach also **reflects real-world scenarios**, as our teaching assistant noted that texts often lack punctuation (e.g., due to OCR errors or missing pages). Therefore, punctuation is intentionally ignored.

3.2.2 Proposed models

The proposed methodologies utilize two specialized BERT-based models, **GuwenBERT** and **SikuBERT**, to perform the classification task. **BERT** (Bidirectional Encoder Representations from Transformers) is a transformer-based architecture designed to process textual data bidirectionally, capturing contextual relationships in both directions. This capability makes BERT well-suited for understanding nuanced patterns in text, including those present in SinoNom/Chinese prose and verse.

BERT models are pre-trained on large corpora, learning general linguistic features, and can be fine-tuned on domain-specific datasets for specialized tasks. The fine-tuning process adapts the model' s pre-trained knowledge to the structural and semantic char-

```
{
    "0": {
        "content": " 洪惟順天年間　稔屢登以至紹平大寶休徵荐至．陛下大政未親亦無過舉而水旱相仍灾🄵屢見．
        ↪  意者臣等不能體陛下仁民愛物之心調變失宜如🄵所云.",
        "label": "prose"
    },
    "1": {
        "content": " 秋燈如孤螢，熠熠耿窗🄵。秋雨如漏🄵，點滴連早暮。我豈楚逐臣，慘🄵出怨句？逢秋未免悲，
        ↪  直以🄵國故。三軍老不戰，比屋困征賦。可使江淮間，歲歲常列戍？",
        "label": "verse"
    },
    "2": {
        "content": " 中書監正字阮竦蔡叔廉．典書潘裹等奉🄵書．金光門待🄵蘇碍奉🄵篆．按杜潤記云帝王政治之
        ↪  大莫急於人材國家制度之詳悉待夫後聖.",
        "label": "prose"
    },
    "3": {
        "content": " 帝但以師稱而不名．嘗撰北江開嚴寺碑文其🄵曰寺廢而興既非吾意．碑立而刻何事吾言．方今
        ↪  聖朝欲暢皇風以振頹俗．🄵端在可黜聖道當復.",
        "label": "prose"
    },
    "4": {
        "content": " 太白騎鯨去，空留采石祠。當軒千里水，繞屋萬松枝。山月長清夜，江雲無盡時。誰將一尊酒，
        ↪  把臂一論詩。",
        "label": "verse"
    }
}
```


Listing 4: Example of the Dataset After Preprocessing for Fine-Tuning Model

acteristics of SinoNom and Chinese texts, enabling it to distinguish prose from verse more effectively.

GuwenBERT

GuwenBERT is a BERT-based model pre-trained on classical Chinese texts. This pretraining makes it particularly adept at understanding the stylistic and structural patterns of traditional prose and verse. Fine-tuning GuwenBERT on the task-specific dataset ensures it adapts to the unique features of SinoNom texts, allowing it to classify text segments while preserving historical and linguistic context.

For our task, we used GuwenBERT-base model, which has about 108 million parameters for our fine-tuning task.

SikuBERT

SikuBERT, on the other hand, is pre-trained on texts from the Siku Quanshu, one of the most comprehensive collections of Chinese literature. This pretraining provides SikuBERT with extensive exposure to a variety of genres, including historical, philosophical, and poetic texts. As a result, SikuBERT offers a broader contextual understanding, making it suitable for classifying texts that may blend multiple styles or exhibit less conventional structures.

For our task, we used SikuBERT model, which has about 103 million parameters for our fine-tuning task.

3.2.3 Fine-tuning methods

Fine-tuning with default hyperparameters

We started by loading pre-trained models and tokenizers for GuwenBERT and SikuBERT to fine-tune them for a classification task, distinguishing between "verse" and "prose" texts. Minimal changes were made to the default hyperparameters. After testing, we found the following training setup produced the most stable results:

- **Epochs:** 3
- **Batch Size:** 16
- **Learning Rate:** Default value as determined by the pretrained models (5e-5).
- The remaining hyperparameters are also kept as default for each proposed model.

Fine-tuning with additional validation dataset

To focus on improving the data rather than tuning model hyperparameters, we introduced a validation dataset. The data was split into three parts:

- Train Set: 80%
- Validation Set: 10%
- Test Set: 10%

The same hyperparameter configuration as the first method was used, as it gave stable results.

Fine-tuning with k-fold cross validation technique

Finally, we applied 5-fold cross-validation to identify the best-performing model. The same hyperparameter setup as in the first method was used for each fold. In both models, the second fold produced the best results and we selected for further evaluation.
Each of the three methods was applied to both GuwenBERT and SikuBERT, resulting in six models for evaluation in the next section (Experiment and Evaluation).

3.2.4 Handling for large input: Majority Voting

When working with lengthy texts, it becomes essential to have a strategy for effectively handling large input data without exceeding the model's maximum token limit. Both GuwenBERT and SikuBERT, as BERT-based models, impose a token limit (commonly 512 tokens). To address this limitation, we implemented a chunking mechanism within our classifier.

Our approach divides the input text into smaller, manageable chunks that fit within the model's maximum token length (currently we set `max_length = 128`). Specifically, the following steps are taken to handle large input:

- Dynamic Chunking: For inputs exceeding the token limit, the text is split into overlapping chunks. Overlapping is introduced to preserve context between consecutive segments, as this is critical for maintaining semantic continuity in texts like verse or prose. Each chunk is designed to be slightly smaller than the maximum length, with a configurable overlap percentage (default: 20%).
- Minimum Chunk Size Enforcement: To avoid excessively small fragments that could disrupt model predictions, a minimum chunk size (50 characters) is enforced. This ensures that each chunk provides sufficient context for accurate classification.
- Prediction at Chunk Level: Each chunk is individually tokenized and passed through the fine-tuned model for classification. The model outputs a label ("verse" or "prose") for each chunk, along with a confidence score.
- Majority Voting for Final Classification: The final classification for the entire text is determined through a majority voting system. Each chunk' s label contributes to the overall decision, weighted by its confidence score. This approach ensures that the model's predictions are robust, even when individual chunks may occasionally yield less confident results.

By employing this strategy, our system effectively processes large input texts, enabling good analysis on lengthy text. This method ensures that the important features of lengthy texts are preserved for model, while remaining compatible with the tokenization constraints.

4 Experiment and evaluation

4.1 Experiment on the test dataset

In this section, we will focus on testing our different methods for two approaches: rule-based and model-based. The methods for two approaches have been illustrated in the previous section. However, as we can see in previous section, for rule-based method, it requires punctuations to classify, so we use both the dataset with punctuations and no punctuations for rule-based. The remaining methods with model-based approach just use dataset with no punctuations and spaces.

Methods	**Precision**	**Recall**	**F1-score**
Rule-based with full punctuations and spaces dataset	0.8282	0.7863	0.7817
Rule-based with no punctuations and spaces dataset	0.2544	0.5000	0.3373
Fine-tuning **GuwenBERT** model with default hyperparameters	0.9912	0.9913	0.9912
Fine-tuning **GuwenBERT** model with additional validation dataset	0.9995	0.9995	0.9995
Fine-tuning **GuwenBERT** model with k-fold cross validation technique	0.9926	0.9927	0.9927
Fine-tuning **SikuBERT** model with default hyperparameters	0.9945	0.9946	0.9945
Fine-tuning **SikuBERT** model with additional validation dataset	**1.0000**	**1.0000**	**1.0000**
Fine-tuning **SikuBERT** model with k-fold cross validation technique	0.9954	0.9955	0.9955

Table 2: Experiment results on the test dataset

Evaluation

- Rule-based methods exhibited relatively good result for the test set. The precision, recall, and F1-score for the rule-based approach with full punctuation and spaces achieved moderately high values (0.8282, 0.7863, and 0.7817, respectively). However, as mentioned before, these metrics dropped considerably when punctuation and spaces were removed, highlighting the dependency of rule-based systems on structured input. This variation underscores the limitations of heuristic methods in handling unstructured or noisy text data.
- The fine-tuned models using GuwenBERT and SikuBERT significantly outperformed the rule-based approaches across all metrics. Fine-tuning GuwenBERT with default hyperparameters yielded a high precision, recall, and F1-score of 0.9912, demonstrating the model's capacity to generalize well to the test dataset. Further improvement was observed when an additional validation dataset was incorporated,

resulting in near-perfect performance (0.9995 for all metrics). The k-fold cross-validation technique provided slightly better performance stability, with marginal increases in precision, recall, and F1-score.

- SikuBERT models consistently better than GuwenBERT models across all the methods. Fine-tuning SikuBERT with default hyperparameters achieved impressive metrics of 0.9945 for precision, recall, and F1-score. The addition of a validation dataset further elevated the model's performance to the highest score (1.0000 for all metrics). Similarly, the k-fold cross-validation strategy demonstrated high performance (0.9954 for precision, and 0.9955 for recall and F1-score.
- In summary, the results clearly highlight the superiority of fine-tuned transformer-based models on the test dataset, which achieves very high performance.

However, it is important to note that the results from this experiment were obtained using the test dataset, which may share similarities with the training dataset. Therefore, further testing on data from different sources and formats is necessary to identify a robust model suitable for this task.

4.2 Experiment on New Samples from External Sources (Internet, OCR Results, etc.)

The unrealistic results from the model-based approach in the previous section may be attributed to the test dataset having patterns similar to the training dataset. To address this, we aim to investigate the accuracy of each method using external sources not included in the training, validation, or test sets. During the semester, we gathered more data from external source for inferencing with each method. Here we provide seven representative data items, each item has an unknown format (e.g., missing punctuations or spaces). These data can be found in the source code folder (file `inference_test.txt`). Beside, we also provide a GUI application. The GUI code can also be found in the source code folder (file `inference_GUI.py`). With this application, user can input their text, choose the desired approach (model-based or rule-based) and get the result predicted. We used this for inferencing data items which will be discussed next.

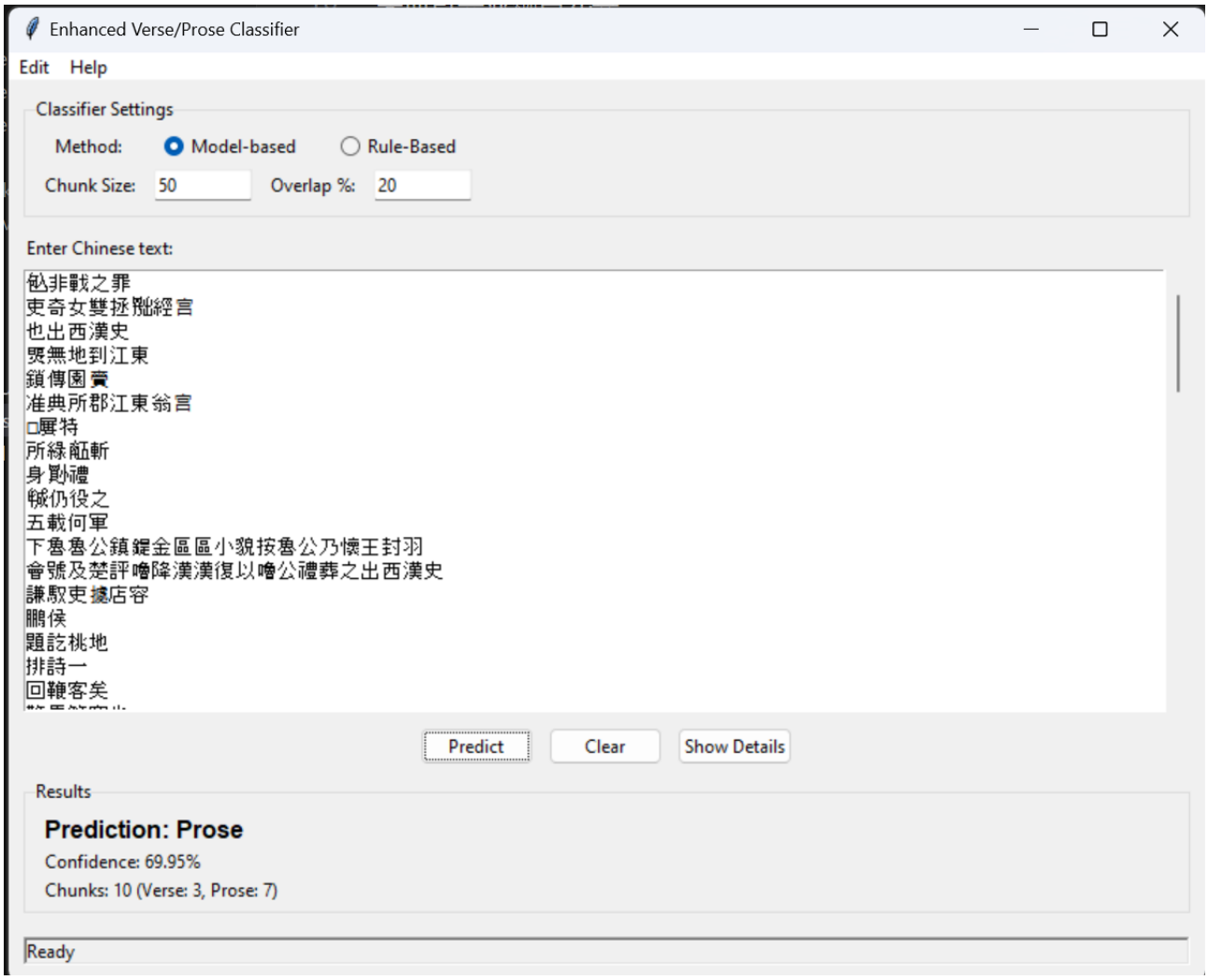

Figure 1: GUI application for user to input text and receive predict result

Below is a description of each data item:

- **Item I:** A segment of four sentences from ”Luc Van Tien Tale,” labeled as ”verse.”
- **Item II:** Data obtained from our teaching assignment. The instructor provided an image of a ”verse,” and we used an OCR tool to retrieve the text.
- **Item III:** Data obtained from our teaching assignment. The instructor provided an image of ”prose,” and we used an OCR tool to retrieve the text.
- **Item IV:** Data obtained from our teaching assignment. The instructor provided an image of ”prose,” and we used an OCR tool to retrieve the text.
- **Item V:** A segment of twelve sentences from ”Chinh Phu Ngam,” labeled as ”verse.”
- **Item VI:** A segment comprising the first 42 lines of ”Binh Ngo Dai Cao,” labeled as ”prose.”
- **Item VII:** A segment comprising the first two paragraphs of ”Hich tuong si,” labeled as ”prose.”

The table below shows the results for each method when inferencing on each data item. If the method correctly predicts the genre of a data item, it is marked as "True"; otherwise, it is marked as "False."

Method \ Data Item	**Data Items**						
	I	**II**	**III**	**IV**	**V**	**VI**	**VII**
Rule-based	False	False	True	True	False	True	True
Fine-tuning **GuwenBERT** model with default hyperparameters	True	True	True	True	True	True	False
Fine-tuning **GuwenBERT** model with additional validation dataset	True	True	False	False	True	False	False
Fine-tuning **GuwenBERT** model with k-fold cross validation technique	True	True	True	True	True	True	False
Fine-tuning **SikuBERT** model with default hyperparameters	True	True	False	True	True	True	False
Fine-tuning **SikuBERT** model with additional validation dataset	True	True	False	True	True	False	False
Fine-tuning **SikuBERT** model with k-fold cross validation technique	True	True	True	True	True	True	False

Table 3: Experiment Results on Different Samples from External Sources

Evaluation

- Although model-based methods achieve nearly perfect scores on the test dataset, their performance on real-world data is less reliable. All methods incorrectly predicted at least one data item.
- The **rule-based** approach, due to missing formatting (e.g., punctuations and spaces), consistently predicts all data items as "prose."
- **Models fine-tuned with additional validation datasets** tend to predict data items as "prose." This may be due to overfitting, where the model learns specific patterns from the validation set and fails to generalize.
- **Models with default hyperparameters** and those fine-tuned using k-fold cross-

validation yield better results, suggesting that the fine-tuning task was effective. K-fold cross-validation provides better outcomes by finding the optimal model across folds.

- The final data item, "Hich tuong si," poses the greatest challenge for the methods. Its tone resembles "verse," but it is labeled as "prose." While most methods misclassified it, the **SikuBERT model with k-fold cross-validation** showed the highest confidence toward predict it as "prose," making it a promising candidate for further investigation.

5 Discussion

5.1 Rule-based Approach

From the experiments above, it is evident that the rule-based approach provides explainable results, as it strictly follows predefined rules to predict the genre. However, when dealing with special types of text, it often fails to process them correctly due to missing factors, such as punctuation and spacing. This limitation makes it challenging to apply this approach to real-world problems involving genre classification of text.

5.2 Model-based Approach

In contrast, the model-based approach shows promising results, achieving near-perfect accuracy on the test set for various methods. However, problems arise when these models are applied to new samples, where some predictions are incorrect. This highlights the fact that these models are not yet robust enough to handle special types of text, such as those derived from OCR, which are prone to overfitting.

To address this, more diverse data sources are needed—particularly data from OCR results—to better evaluate and improve the model. Unfortunately, our group faced certain limitations. Due to a lack of expertise (we cannot reliably predict the genre of Chinese/Sinonom texts ourselves) and time constraints (limited to one semester), we focused on improving the model by fine-tuning it with textual data from trusted internet sources, where the genre of the text is accurately provided.

In the future, we hope to construct an OCR-based corpus with labeled data to further improve the accuracy of our models. This will enable the models to handle a wider range of situations when processing text inputs.

6 Conclusion

This project provided us with a valuable and engaging experience in tackling a real-world problem. Throughout this work, we explored various approaches and methods, achieving some promising results. However, due to limitations in computational resources and knowledge, we were only able to produce results at an acceptable level.

We hope to have more opportunities in the future to continue working on this topic and further refine our solutions. We would like to express our gratitude to Professor Dinh Dien and the teaching assistants for giving us the opportunity to deepen our understanding of language, learn how to construct a corpus for a specific purpose, and develop skills to achieve the best results as possible.

7 Reference

1. GuwenBERT
2. SikuBERT